

CCIS

كلية علوم الحاسب والمعلومات
COLLEGE OF COMPUTER &
INFORMATION SCIENCES



Flutter

جامعة الأمير سلطان
PRINCE SULTAN
UNIVERSITY



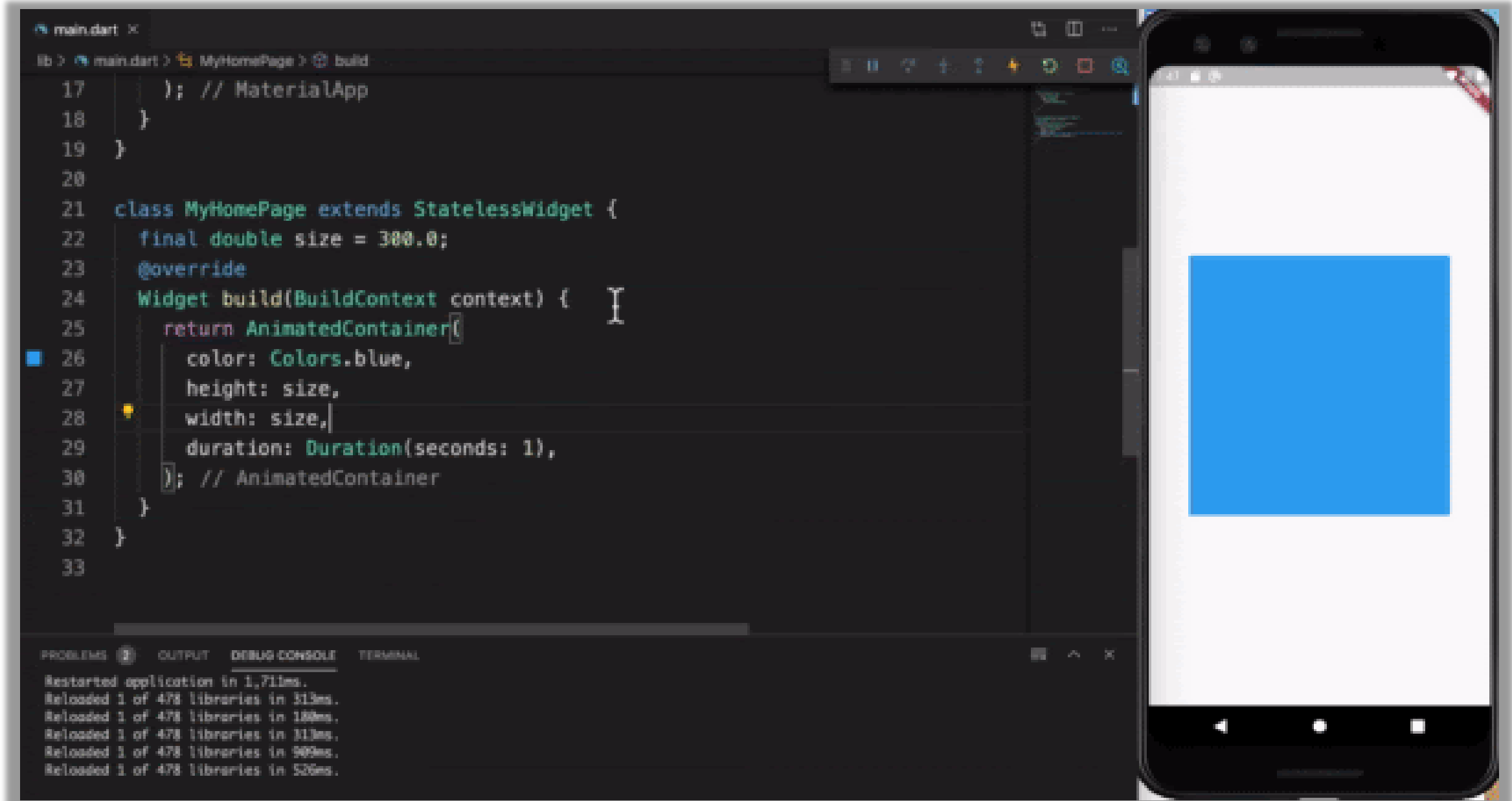
IS487 Emerging Topics in IS

Dr. Abbas Malik
amaalik@psu.edu.sa

Animations

Implicit Animations

- Rely on simply setting a new value for some widget property
- Flutter takes care of animating it from the current value to the new value.
- Easy to use and are amazingly powerful



Implicit Animation

- AnimatedAlign
- AnimatedContainer
- AnimatedPadding
- AnimatedDefaultText
Style
- AnimatedPositioned
- AnimatedOpacity
- AnimatedPositionedDi
rectional
- AnimatedTheme
- AnimatedSize

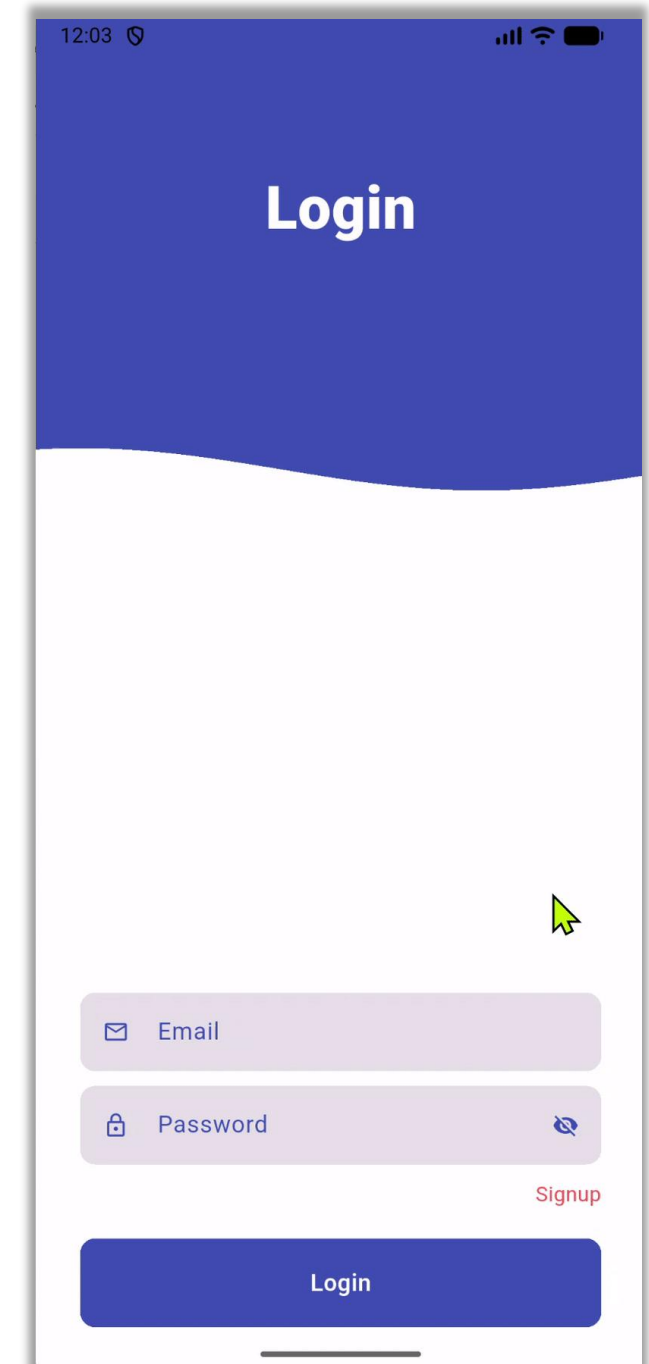
Animations

Explicit Animations

- Require an AnimationController
- Only start animating when explicitly asked to
- Manually manage life-cycle of AnimationController
 - Put in stateful widget and dispose off at the end

Login & Signup App

- Provider: State Management
- Explicit Animation



Issues in Data Storage



Data is stored in the App State and is lost when App is closed



No persistent storage of data



Local Storage as (key, value) pair



SQLite local DB Storage



Cloud Storage like Firebase

Issues in Pages

- Handling everything in one page/screen
- It is better to have different screens for each operation
 1. Login Page
 2. Signup Page
 3. Logged In Page - with services for authorized user

Navigation and Routing



Navigation between screens and handling deep links



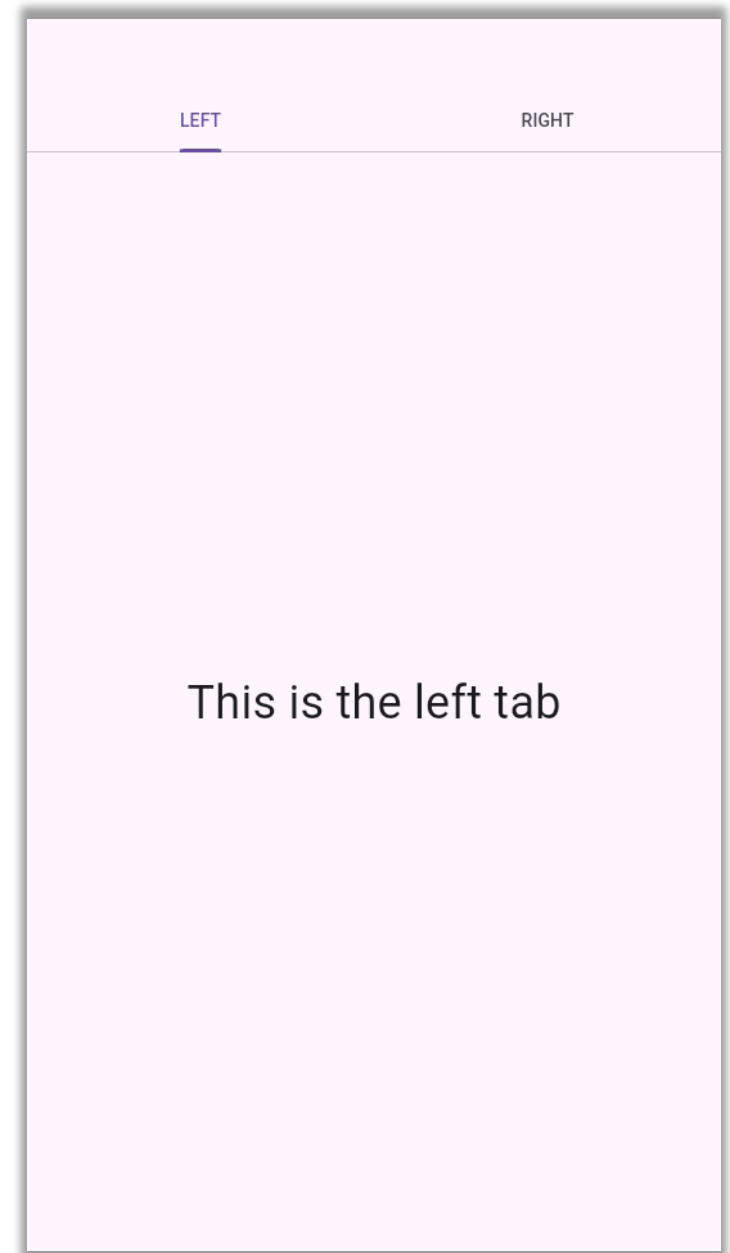
Simple Linking use [Navigator](#)



Complex Navigation and Deep Linking use [Router](#)

Tab Layout

- Multiple Tabs in one page
- Control Tabs with TabController
- Create each Tab
 - Create contents for each Tab

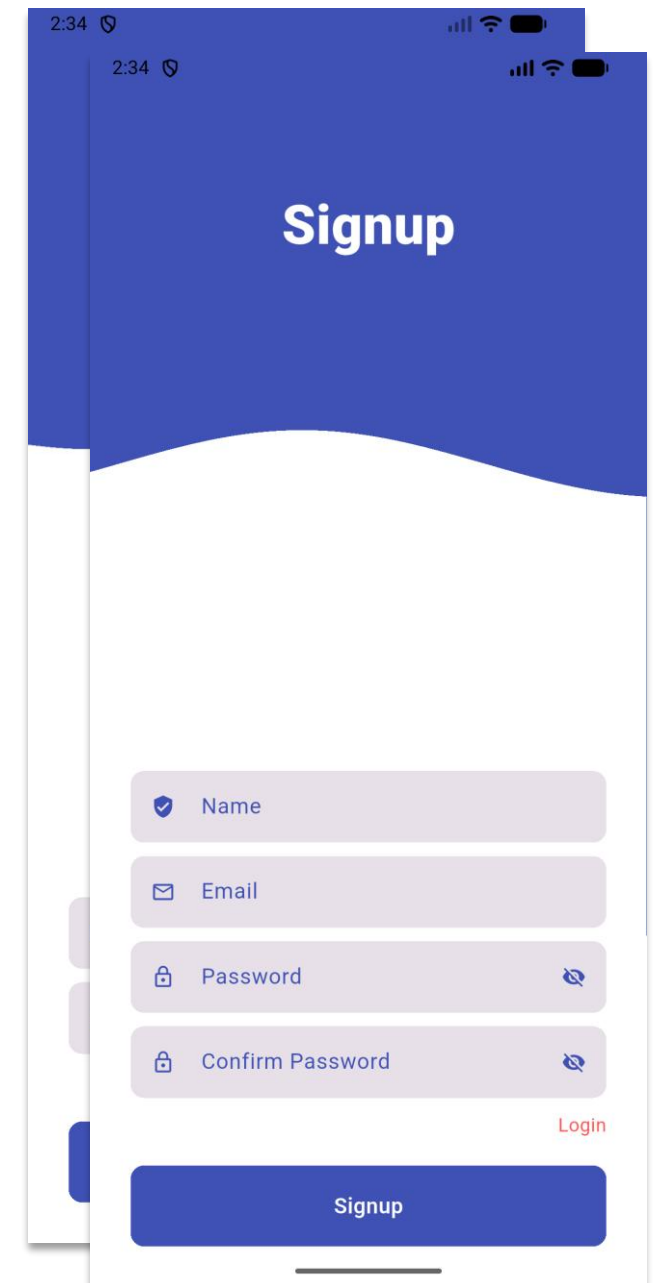


Navigation with Navigator

- Stack pages on top of each other

```
Navigator.of(context).push(  
  MaterialPageRoute<void>(  
    builder: (context) => const SignupPage(),  
  ),  
);
```

```
Navigator.of(context).pop(context);
```



Persistence Local Storage

- Package: localstorage
- Save key-value pair
 1. Add dependency
 2. Save data
 3. Read data
 4. Remove data

Data Persistence - localstorage

- Primitive types are supported: int, double, bool, String and List<String>
- Not designed for large or sensitive data

Class UserModel

Serialization

- Serialize Class into JSON string

1. Manual using
dart:convert package
2. Automated serialization
json_annotation

```
class UserModel {  
  String? _name;  
  String? _email;  
  String? _password;  
  String? get name => _name;  
  set name(String value) {  
    _name = value;  
  }  
  String? get email => _email;  
  set email(String value) {  
    _email = value;  
  }  
  String? get password => _password;  
  set password(String value) {  
    _password = value;  
  }  
}
```

Manual Serializing UserModel class

```
UserModel.fromJson(Map<String, dynamic> json)  
  : _name = json['name'] as String,  
    _email = json['email'] as String,  
    _password = json['password'] as String;
```

```
Map<String, dynamic> toJson() => {  
  'name': _name,  
  'email': _email,  
  'password': _password,  
};
```

```
UserModel() {  
  _name = "";  
  _email = "";  
  _password = "";  
}
```

Manual Serializing UserModel class

Converting UserModel Object $\longleftrightarrow$ JSON String

```
import 'dart:convert';
```

```
String userJsonStr = jsonEncode(user);
```

```
final userMap = jsonDecode(userJsonStr) as Map<String, dynamic>;  
UserModel userObj = UserModel.fromJson(userMap);
```


Saving Object in Storage

```
bool addUser(String name, String email, String password) {  
    if (!(_isExistingUser())) {  
        UserModel user = UserModel();  
        user.name = name;  
        user.email = email;  
        user.password = password;  
        String userJsonStr = jsonEncode(user);  
        storage.setItem('user_info', userJsonStr);  
        return true;  
    } else {  
        return false;  
    }  
}
```

Reading and Checking Object from Storage

```
bool _isExistingUser(String email) {  
    String? userJsonString = storage.getItem('user_info');  
    if (userJsonString != null) {  
        return true;  
    }  
    return false;  
}
```

Reading Object from Storage

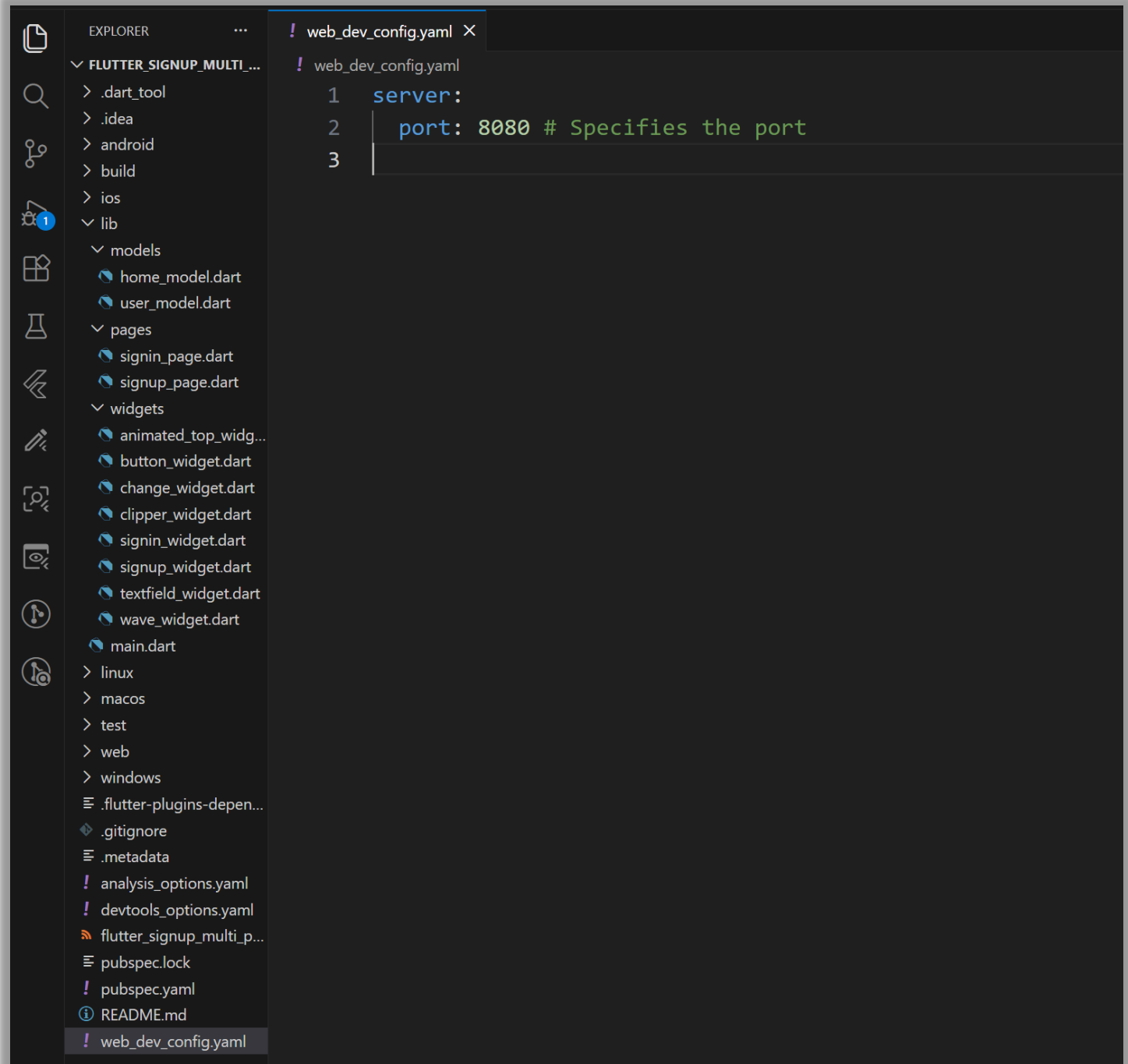
```
UserModel? getValidUser(String email, String password) {  
    String? userJsonString = storage.getItem('user_info');  
    if (userJsonString != null) {  
        Map<String, dynamic> userMap = jsonDecode(userJsonString);  
        if (email == userMap['email'] && password == userMap['password']) {  
            return UserModel.fromJson(userMap);  
        }  
    }  
    return null;  
}
```

Persisting Data between App Restart on Web

- Every time app runs on a different port number
- It prevents the previous data to be lost
- To fix add a `web_dev_config.yaml` file at root of your project and following contents:

```
server:  
  port: 8080 # Specifies the port
```

Persisting Data between App Restart on Web



Passing Data Between Pages/Screens

```
class HomePage extends StatelessWidget {  
  const HomePage({super.key, required this.user});  
  final UserModel user;  
  
  @override  
  Widget build(BuildContext context) {  
    return Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [Text(user.name), Text(user.email), Text(user.password)],  
      ), // Column  
    ); // Center  
  }  
}
```

Passing Data Between Pages/Screens

```
void _signInUser() {  
    // get email and password to verify the user  
    String email = emailController.text;  
    String password = passwordController.text;  
    final UserModel? user = context.read<HomeModel>().getValidUser(  
        email,  
        password,  
    );  
    if (user != null) {  
        emailController.clear();  
        passwordController.clear();  
        Navigator.of(context).push(  
            MaterialPageRoute<void>(builder: (context) => HomePage(user: user)),  
        );  
    } else {  
        ScaffoldMessenger.of(context).showSnackBar(  
            SnackBar(  
                content: Text('Sorry! You failed to logged in. Please try again!'),  
                duration: Duration(seconds: 2),  
                width: 300.0,  
                padding: EdgeInsets.symmetric(horizontal: 10.0),  
                behavior: SnackBarBehavior.floating,  
                shape: RoundedRectangleBorder(  
                    borderRadius: BorderRadius.circular(12.0),  
                ), // RoundedRectangleBorder  
            ), // SnackBar  
        );  
    }  
}
```

Other Packages for Local Storage

Shared_Preferences

- https://pub.dev/packages/shared_preferences

Secure Storage

- https://pub.dev/packages/flutter_secure_storage

State Management

1. **setState()**: The built-in approach for handling ephemeral state within a StatefulWidget
2. Provider: A simple yet powerful solution that uses InheritedWidget to efficiently pass state down the widget tree - **small scale app**

State Management

3. Riverpod: a more robust and compile-time safe alternative that removes the reliance on BuildContext to access providers. It is the modern default for most projects due to its performance, testability, and low boilerplate code. - medium to large scale apps

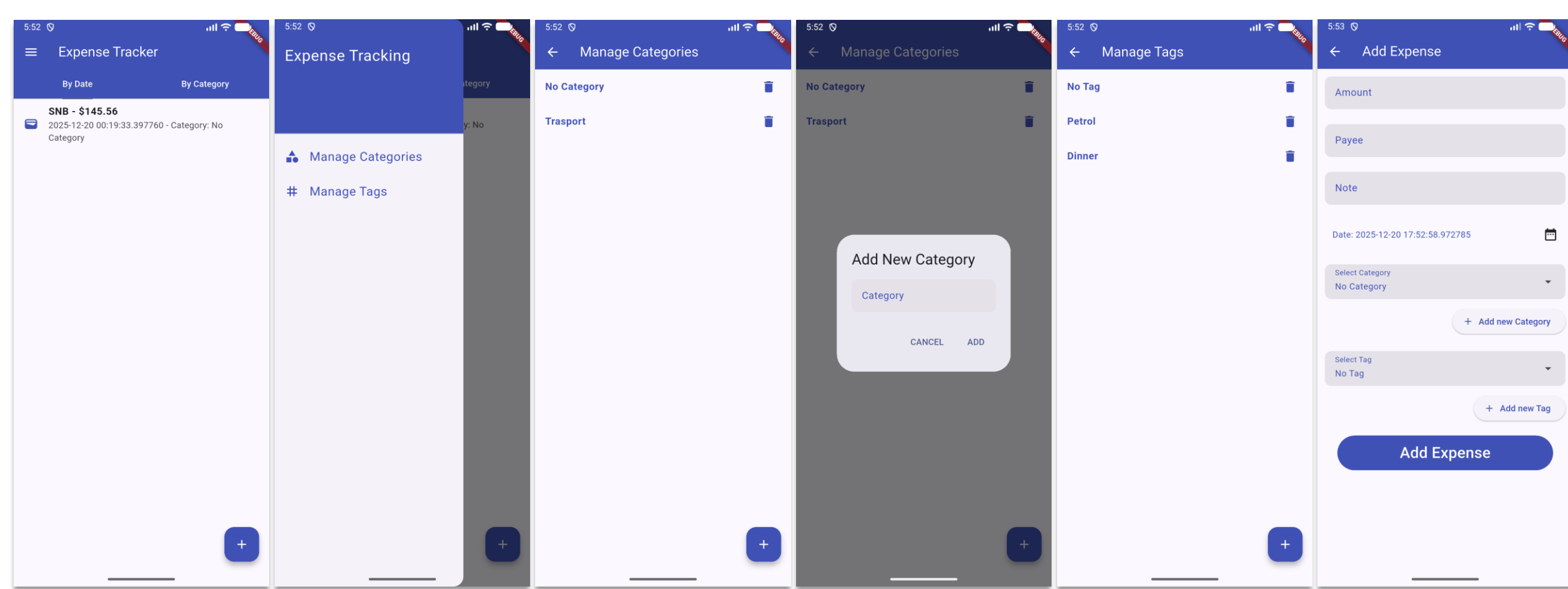
State Management

4. BLoC (Business Logic Component): pattern enforces a strict separation between UI and business logic, using events (inputs) and states (outputs) via streams.
- Makes code **predictable** and **highly testable**
 - Making it an enterprise standard for large, complex applications

Persisting Data

1. Local Storage
2. Local Database SQLITE - sqflite
3. Cloud Database/Storage/NoSQL
4. REST APIs (<http://www.myapi.com/books/>)

Expense Tracking App



References

- <https://docs.flutter.dev/ui/animations>
- [How to choose](#)
- <https://github.com/gdrtrts/animated-login/tree/master>
- <https://docs.flutter.dev/ui/widgets/animation>
- <https://docs.flutter.dev/cookbook/persistence/key-value>
- <https://docs.flutter.dev/data-and-backend/serialization/json>